

Programmation de Spécialité

Rapport du *Projet Python – Moteur de Recherche*

41LIDC01

Professeur : Walid Bechkit

Eoin Brereton Hurley

January 21, 2026



INSTITUT
de La
communication



1 Introduction

Ce rapport présente la conception et l'implémentation d'un moteur de recherche en *Python* dans le cadre du *Projet Python* du cours Programmation de Spécialité. L'objectif principal de ce projet est de permettre la recherche rapide et pertinente d'informations dans un corpus de textes divers, en utilisant une approche basée sur la pondération TF-IDF et la similarité cosinus entre documents et requêtes.

Le projet est organisé en modules Python distincts, chacun correspondant à un concept clé : la gestion des documents, l'organisation des corpus, et le moteur de recherche. Une interface graphique complète permet à l'utilisateur de saisir ses requêtes, de filtrer les résultats par auteur ou par date, et de visualiser les résultats de manière intuitive et interactive.

L'intégralité du code est disponible dans le dépôt *GitHub* suivant :
https://github.com/EoinBH/projet_M1_PDS

2 Spécifications

- Le logiciel a été conçu pour effectuer plusieurs fonctions essentielles. Tout d'abord, il doit être capable de lire un corpus de documents provenant de différentes sources, telles que des fichiers texte, (*Reddit*, *ArXiv* ou *.csv*), et de stocker toutes les métadonnées associées à chaque document. Ensuite, le programme doit nettoyer et découper ces textes en phrases suffisamment longues pour être exploitables lors des recherches.
- Le moteur doit également construire automatiquement un vocabulaire complet et les matrices de fréquence TF et TF-IDF à partir des documents fournis. Il doit être capable de rechercher efficacement des documents contenant des mots-clés, et de calculer la similarité cosinus pour classer les documents par pertinence.
- Le programme doit permettre de rechercher efficacement des documents à partir de mots-clés, de calculer leur pertinence à l'aide de la similarité cosinus, et de filtrer les résultats selon différents critères (auteur, année de publication). Deux modes d'exécution sont proposés : un mode *analyse*, destiné à l'exécution de tests complets et à l'affichage des résultats dans la console, et un mode *utilisateur*, qui fournit une interface graphique interactive développée avec *Tkinter*.

La bibliothèque *Tkinter* a été choisie pour implémenter l'interface graphique au lieu de passer par les *ipywidgets* de *Jupyter Notebook*.

2.1 Calcul de la similarité cosinus

Pour mesurer la pertinence d'un document par rapport à une requête, le moteur de recherche utilise la **similarité cosinus**. Cette mesure compare l'angle entre le vecteur représentant la requête **q** et le vecteur représentant un document **d** dans l'espace des termes pondérés par TF-IDF.

La formule mathématique est la suivante :

$$\text{similarité_cosinus}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}} \quad (1)$$

où :

- q_i est la valeur TF-IDF du terme i dans la requête,
- d_i est la valeur TF-IDF du terme i dans le document,
- n est le nombre total de termes dans le vocabulaire.

Dans le code Python, cette opération est réalisée par la méthode `_similarite_cosinus` de la classe `SearchEngine`. Le produit scalaire entre la requête et chaque document est calculé à l'aide de `mat_TFIDF.dot(vecteur_requete.T)`, puis divisé par le produit des normes des vecteurs pour obtenir le cosinus. Les valeurs NaN sont remplacées par zéro pour éviter les erreurs en cas de documents vides.

Pour exécuter le programme, il suffit de lancer la commande suivante dans un terminal :

```
1 python3 main.py
```

L'utilisateur est alors invité à choisir le mode d'exécution souhaité : tests complets ou interface graphique.

2.2 Bibliothèques

Le programme utilise plusieurs bibliothèques Python : **numpy** pour les calculs scientifiques, **Tkinter** pour l'interface graphique, **re** pour le traitement des expressions régulières, **pandas** pour la lecture et la manipulation de données, **datetime** pour la gestion des dates, **scipy.sparse** pour la manipulation de matrices creuses, **tqdm** pour visualiser la progression de certaines opérations, ainsi que **praw**, **urllib.request**, **ssl** et **certifi** pour l'accès aux données **Reddit** et à d'autres sources web. La bibliothèque **tabulate** est utilisée de manière optionnelle pour formater les sorties tabulaires.

3 Analyse

L'environnement de travail choisi pour ce projet est *Python 3.11*, utilisé avec l'IDE *VS Code*, qui offre une bonne intégration des bibliothèques scientifiques et un débogage facile. Le corpus principal exploité est le fichier CSV `discours_US.csv`, qui contient les colonnes `speaker`, `text`, `date` et `link`, correspondant respectivement à l'orateur, au texte du discours, à la date de publication et au lien source du document.

Diagramme conceptuel simplifié :

Le projet repose sur plusieurs classes principales. La classe `Document` est responsable du stockage du texte et de ses métadonnées, tandis que ses sous-classes `RedditDocument` et `ArxivDocument` ajoutent des attributs spécifiques à chaque type de document. La classe `DocumentFactory` centralise la création des documents selon leur type, et la classe `Corpus` gère la collection complète des documents tout en offrant des fonctionnalités de recherche et de concordancier. Enfin, le moteur de recherche `SearchEngine` exploite les matrices TF-IDF pour effectuer des recherches par similarité cosinus.

3.1 Diagramme des classes

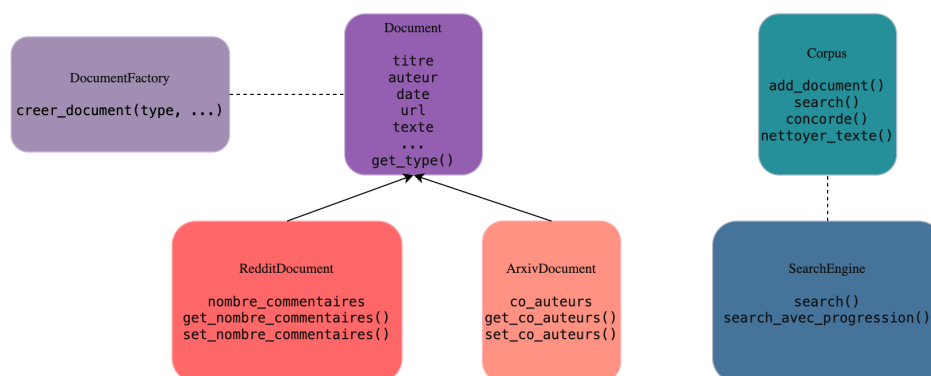


Figure 1: Diagramme simplifié des classes du moteur de recherche

Les relations principales identifiées dans le diagramme sont les suivantes. Les classes `RedditDocument` et `ArxivDocument` héritent de la classe `Document`, ce qui leur permet de réutiliser les attributs et méthodes de la classe parente. La classe `Corpus` utilise les classes `Document` et `DocumentFactory` pour gérer les documents et effectuer des recherches. Le moteur `SearchEngine` exploite

le `Corpus` pour calculer les similarités et retourner les résultats pertinents.

4 Conception

La conception a été pensée de manière modulaire afin de faciliter l’extension et la maintenance du code. Chaque module correspond à un aspect particulier du projet, ce qui permet de travailler de manière indépendante sur chaque composant.

Deux pipelines de construction de corpus ont été implémentés. Le premier permet de collecter automatiquement des données à partir de `Reddit` et d’`ArXiv` en fonction d’un thème donné, tandis que le second exploite un corpus local de discours politiques stocké sous forme de fichier CSV. Cette séparation permet de comparer des sources textuelles hétérogènes tout en réutilisant le même moteur de recherche.

4.1 Modules principaux

Le module `Document.py` contient la classe `Document` ainsi que ses spécialisations pour `Reddit` et `ArXiv`. Le module `DocumentFactory.py` est chargé de créer des objets `Document` selon leur type. Le module `Corpus.py` gère l’ensemble des documents, fournit des méthodes pour nettoyer les textes et créer un concordancier. Le module `SearchEngine.py` implémente la construction des matrices TF et TF-IDF, le calcul de similarité cosinus, et le classement des résultats. Enfin, le script `main.py` constitue le point d’entrée unique du programme : il initialise le corpus, crée le moteur de recherche et lance l’interface `Tkinter`. Un mécanisme interactif permet à l’utilisateur de choisir dynamiquement entre un mode test et un mode interface, ce qui rend le programme à la fois exploitable pour l’analyse et pour une utilisation finale.

4.2 Algorithmes clés

Le programme commence par découper les textes en phrases suffisamment longues pour éviter le bruit dans les résultats. Chaque phrase est ensuite nettoyée pour supprimer les caractères spéciaux et normaliser le texte. Le vocabulaire est construit à partir de l’ensemble des mots présents dans le corpus, et les matrices TF et TF-IDF sont générées pour mesurer l’importance de chaque terme dans chaque document. La recherche par similarité cosinus permet de classer les documents en fonction de leur pertinence par rapport à une requête donnée, et l’interface graphique permet ensuite de filtrer les résultats selon l’auteur ou la date de publication.

Exemple d'utilisation :

Tout d'abord, le corpus CSV est chargé et transformé en objets `Document`. Ensuite, le moteur TF-IDF est construit via la commande `moteur = SearchEngine(corpus)`. L'interface Tkinter peut ensuite être lancée avec la fonction `lancer_interface(moteur)`, permettant à l'utilisateur de saisir des requêtes, d'appliquer des filtres et de visualiser les résultats.

4.3 Problèmes rencontrés et solutions

Plusieurs problèmes ont été identifiés au cours du développement. La présence de dates manquantes ou mal formatées a été résolue en utilisant une conversion sécurisée avec une valeur par défaut lorsque le format de date n'est pas reconnu. Certaines phrases du corpus étaient trop courtes ou bruitées, ce qui a été contourné en fixant un seuil minimal de 20 caractères pour la prise en compte dans le moteur de recherche. Enfin, la gestion du vocabulaire et des matrices a été optimisée en utilisant un dictionnaire pour stocker les fréquences TF et DF, puis en triant et indexant les mots afin d'améliorer la performance du moteur.

5 Validation

5.1 Tests unitaires

Chaque méthode clé a été testée individuellement. Le calcul des fréquences TF et DF pour chaque mot a été vérifié, et la construction des matrices TF-IDF a été validée. Les méthodes `Corpus.search` et `Corpus.concorde` ont été testées pour s'assurer qu'elles retournent les résultats corrects.

5.2 Tests globaux

Des tests ont été réalisés en utilisant l'interface graphique. Les recherches ont été effectuées avec des mots-clés présents ou absents dans le corpus afin de vérifier le comportement du moteur. Le filtrage par auteur et par année a été testé pour confirmer que les critères sont correctement appliqués, et le classement des documents selon le top N résultats les plus pertinents a également été vérifié.

6 Maintenance

Le logiciel a été conçu selon une architecture modulaire facilitant sa maintenance et son évolution. La séparation claire entre les modules de gestion des documents, du corpus, du moteur de recherche et de l'interface graphique

permet d'enrichir le projet sans remettre en cause l'existant.

Toutefois, par manque de temps, il n'a pas été possible d'implémenter l'ensemble des fonctionnalités prévues dans le cadre des TD 9-10. Ces derniers visaient à approfondir l'exploration visuelle et comparative des corpus, notamment en proposant des outils destinés à des utilisateurs non informaticiens. Parmi les extensions envisagées figuraient la comparaison de plusieurs corpus (par exemple **Reddit** et **ArXiv**), l'analyse de l'évolution temporelle de certains termes, ainsi que l'étude de l'importance relative des mots à l'aide de mesures avancées telles que TF-IDF adapté ou Okapi-BM25.

De même, l'interface graphique aurait pu être enrichie par des visualisations comparatives, des filtres plus avancés et des représentations temporelles du vocabulaire. Bien que ces fonctionnalités n'aient pas pu être développées, l'architecture actuelle du projet constitue une base solide permettant leur intégration ultérieure sans modifications majeures.

Facilité de maintenance : La structure modulaire et l'utilisation du pattern `DocumentFactory` permettent d'ajouter facilement de nouveaux types de documents, d'appliquer des filtres supplémentaires, et de modifier ou enrichir le moteur de recherche sans impacter le code existant.

7 Conclusion

Ce projet a permis de développer un moteur de recherche textuel performant basé sur le calcul TF-IDF et la similarité cosinus. L'ensemble des fonctionnalités implémentées permet de construire un vocabulaire complet à partir de documents variés, de réaliser des recherches pertinentes par mots-clés, de filtrer les résultats par auteur et par date, et de visualiser les résultats grâce à une interface graphique intuitive.

Le projet est facilement extensible à de nouveaux types de documents et à des fonctionnalités plus avancées, telles que l'export des résultats ou l'intégration d'un moteur plus sophistiqué. Les principes de modularité et de réutilisation du code facilitent également la maintenance et l'évolution future de l'application.